

Zeitaufwandsanalyse - OpenSqueakMaps Geoguessr

7 Personen × 5-6h (ohne Vorlesungen und Tutorien) × 8-9 Wochen → grob 280-360h brutto
abzüglich Meetings, Orga, Reviews, ggf. Einarbeitung Smalltalk/Squeak → realistisch 240-290h netto Entwicklungszeit

wichtig zu erwähnen: Foto-Sammlung des Campus ist eigene Aufgabe, die nicht „Code“ ist, aber Zeit kostet (Begehung, Auswahl, Geo-Referenzieren). Mindestens 10-15h sollten dafür eingeplant werden, bevor das Spiel überhaupt sinnvoll testbar ist.

Was wir auf jeden Fall umsetzen können (Core)

1. Anzeige von Fotos vom Campus — ~25-35h

Datenmodell für Foto + Koordinate + Metadaten (SMGFoto o.ä.)

Asset-Strategie: Fotos im Squeak-Image vs. extern geladen — Image-Größe wird sonst schnell unhandlich

Photo-Morph (Skalierung, Rahmen, evtl. Zoom-Funktion fürs Bild)

Sammelobjekt + Persistenz (welche Fotos gibt's, wo gespeichert)

Risiko: Squeak ist beim Laden vieler/großer Bilder eher zäh — JPEG-Decoding und Form-Verwaltung können Bottleneck werden, wenn man's naiv macht

2. Karte mit Marker setzen (über bestehendes Maps-Feature) — ~15-25h

SqueakMaps als Basis einbinden (SMAApplication adaptieren oder Submorph nutzen)

Pin-Mechanik existiert bereits (buildSearchPinPanel, SMAMapConfigurator displayLocation:) → kann recycelt werden

Map-Ausschnitt auf Campus festklemmen (Bounding Box + Zoom-Limits)

Tile-Source sinnvoll wählen (OSM ohne Key am robustesten, Thunderforest nur falls Style erwünscht)

Click-Event auf der Karte zu Map-Koordinate umrechnen — die Logik ist im Repo vorhanden, muss aber für „Guess“ wiederverwendet werden

3. Guess-Mechanik — ~20-30h

UI-Workflow: Foto links/oben, Karte daneben, „Bestätigen“-Button

Klick auf Karte setzt vorläufigen Guess-Pin → bestätigen lockt den Guess fest

Reveal-Phase: echter Pin + Linie zwischen Guess und Wahrheit (Polyline auf der Karte zeichnen — Pattern existiert bereits in displayRoute:)

Distanzberechnung (bei Campus-Größe reicht ebene Geometrie, kein Haversine nötig)

States sauber halten: „guessing“ / „locked“ / „revealed“ — sonst handelt sich's schnell nervige Bugs ein

4. Punktberechnung basierend auf Distanz und Zeit — ~15-25h

Punkte-Formel + Tuning (z.B. $5000 - \text{distance} \cdot a - \text{time} \cdot b$, mit Schwellen)

Timer pro Runde (Squeak Time millisecondClock reicht, aber sauber pausierbar machen falls Reveal-Phase)

Anzeige der Punkte nach Runde (Animation optional, kostet aber gefühlt viel mehr Zeit als man denkt — lieber statisch zuerst)

Edge Cases: Timeout ohne Guess → 0 Punkte; Zoom-Niveau beeinflusst Distanz nicht

5. Einfache Rundenlogik — ~20-30h

Spielzustand als State Machine (SMGGame mit #waiting #playing #revealing #ended)

Bildauswahl-Strategie (Shuffle ohne Wiederholung, fixe Rundenzahl wie 5)

Übergänge zwischen Runden (Reveal → kurze Pause → nächstes Foto laden, dabei Map zurücksetzen via clear)

End-Screen mit Gesamtpunktzahl + „Nochmal spielen“

saubere Reset-Logik beim Neustart (alle Pins weg, Timer zurück, States zurück) — typische Bug-Quelle

Querschnittsaufgaben — ~50-80h

Setup, Package-Struktur, Anbindung an existierendes SqueakMaps: 15-20h

Tests (mit MorphicTestingFramework, das ist ja eh Dependency): 20-30h

Bugfixing + Polishing: 15–20h

Code Reviews, CI grün halten, Linter durchziehen: 10h

Einarbeitung Smalltalk/Squeak (falls nicht alle erfahren sind): zusätzlich 2–4h pro Person, je nach Vorkenntnis

Summe Core: ~145–225h

→ Passt komfortabel in die 240–290h, lässt also Puffer für Stretch-Features oder Krisenmanagement.

Was wir wahrscheinlich zusätzlich schaffen könnten

Einfaches Leaderboard — ~15–25h

Persistenz: Squeak-Image kann das via Smalltalk saveSession halten, oder klassisch STON/ FileDirectory-Datei

Sortierung + Anzeige (Top N, Datum, Spielername)

Eingabe des Spielernamens

Wenn lokal, ist's überschaubar — wenn übers Netz geteilt werden soll, fällt's in die kritische Kategorie unten

UI-Verbesserungen — ~15–25h

Hover-Effekte, Übergänge zwischen States, bessere Pin-Icons

Sounds beim Guess/Reveal (optional)

Konsistentes Styling am Styleguide entlang

Ehrlich gesagt: das schluckt schnell mehr Zeit als geplant, weil Morphic in vielen Detail-Stellen umständlich ist

„Mobile Nutzung über Browser“ — wahrscheinlich eher ein nice to have, da nicht wirklich sinn von Squeak, aber könnte man mit genügend Zeit ausprobieren

Was kritisch / aufwendig würde

Multiplayer (Räume + Passwort, synchron) — ~60–100h+

Serverkomponente: entweder eigener Squeak-Server (mit z.B. WebSockets über Zinc o.Ä.) oder externer Service. Beides bedeutet Netzwerk-Stack lernen, deployen, debuggen.

Sync-Protokoll: Spielzustand zwischen N Clients konsistent halten — wer ist „dran“? Was passiert bei Disconnect? Wie reveal man synchron?

Race-Conditions: zwei Clients senden gleichzeitig Guess → wessen Zeit zählt?

Test-Aufwand explodiert: Multi-Client-Tests sind aufwendig zu schreiben

→ easy 30–40% der gesamten Projektzeit, oft ohne dass das Kernspiel davon besser wird.

Maximal in einer abgespeckten Variante (asynchron: alle spielen die gleichen Bilder, vergleichen Punkte am Ende — das wäre eher ~20–30h).

StreetView-ähnliche Lösung — nicht realistisch

Photogrammetrie / 360°-Aufnahmen + Player + Navigation: locker 150h+ allein für die Engine, ohne Content

Sphärische Bildanzeige in Morphic existiert nicht out of the box

Gebäude mit mehreren Etagen als Layer — ~30–60h

Eigene Map-Layer-Logik (UI für Etagen-Switch, eigene Tile-Sources je Etage oder Overlay-Bilder)

Datenpflege je Etage (welche Fotos gehören wohin)

Nicht Kern-Gameplay → nice to have, aber eher Phase 2

Mögliches Wochen-Setup:

Wochen 1–2: Setup, Foto-Sammlung (parallel!), Datenmodell, SqueakMaps integrieren, einfache Karte + Marker

Wochen 3–5: Guess-Mechanik + Punkte + Rundenlogik → spielbares MVP nach Woche 5

Woche 6: Bugfixing, Tests, Polishing → stabile Version

Wochen 7–8: Stretch nach Priorität (Leaderboard zuerst, dann UI-Polishing), Puffer für unerwartete Probleme

Woche 9: Abschluss, Doku, Demo-Vorbereitung

Konkrete Verzichtsempfehlungen:

Multiplayer: streichen oder durch asynchronen Score-Vergleich ersetzen

3D/StreetView: streichen

Mehretagen-Layer: als Phase 2 kennzeichnen

Mobile-Browser: klären, ob SqueakJS akzeptabel ist; sonst streichen

Goldene Regel: lieber Woche 5 ein wirklich rundes, spielbares Spiel haben und dann 2–3 Wochen polieren, als Woche 8 noch am Multiplayer-Sync zu debuggen.